Formulários: construir para entender

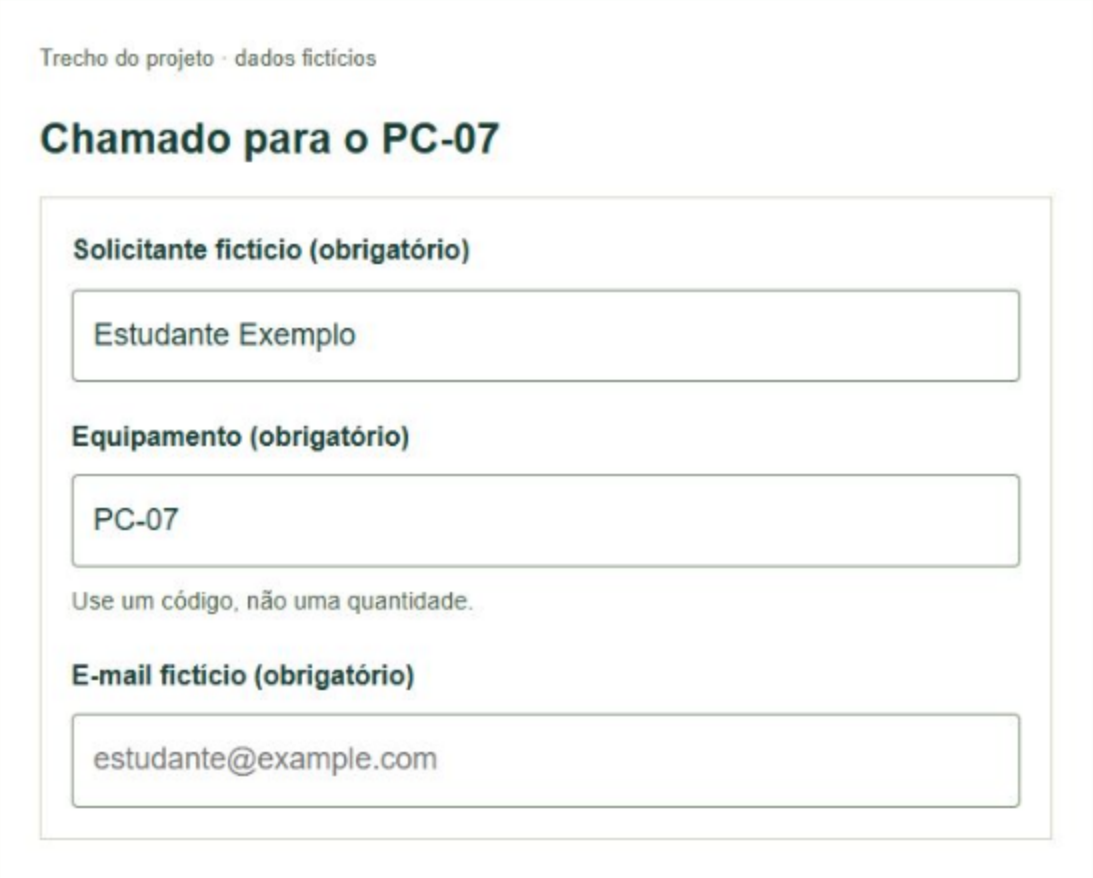
Do pedido de ajuda ao primeiro formulário.
HTML hoje; CSS e JavaScript nos próximos encontros.

Observe → construa → salve → teste → explique

Uma pessoa precisa pedir ajuda

O computador PC-07 está lento. Quais informações ajudam o suporte?

Observe o formulário e relacione cada campo a uma necessidade.



Qual controle combina com o dado?

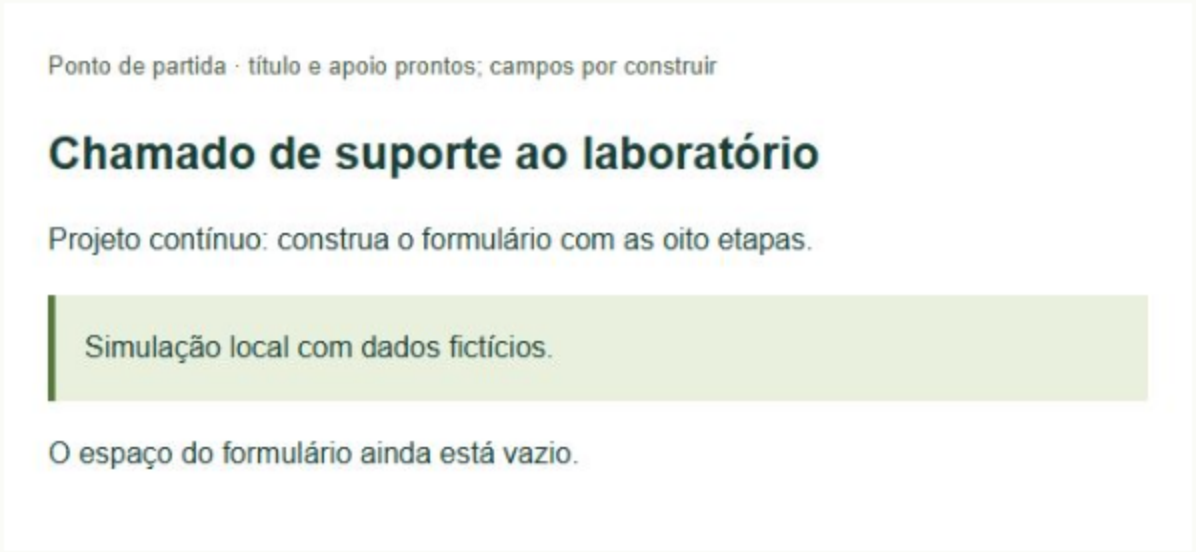
Texto curto	Texto longo	Uma escolha	Várias escolhas
input	textarea	select / radio	checkbox

Texto curto → input. Texto longo → textarea.
Uma opção → select ou radio. Várias opções → checkbox.

Agora: abrir a base no VS Code

Baixe e extraia o projeto da aula 02. Abra a pasta e use Live Server em index.html.

A página começa sem campos. Hoje vamos editar somente index.html.



Tag cria; atributo configura

```
<label for="contato">E-mail fictício</label>
<input id="contato" name="contato"
  type="email" required>
```

label é a tag. for é um atributo. Seu valor precisa coincidir com o id do input.
name identifica o dado; type escolhe o comportamento.

1 · Criar o formulário

index.html: substitua o comentário INÍCIO DO FORMULÁRIO dentro de main.

```
<form id="formulario">
  <div class="campos">
    <!-- CAMPOS: inserir aqui -->
  </div>
  <!-- GRUPOS: inserir aqui -->
  <!-- BOTÃO: inserir aqui -->
</form>
```



Trecho para o quadro · bloco completo no portal

Confira que existe um único form, com abertura e fechamento.

2 · Identificar quem pede ajuda

index.html: antes do comentário CAMPOS, dentro de div.campos. Mantenha o comentário para inserir os próximos campos.

```
<label for="solicitante">Solicitante fictício</label>
<input id="solicitante" name="solicitante"
  type="text" required
  minlength="2" maxlength="60">
```



Trecho para o quadro · bloco completo no portal

Clique no rótulo: o cursor deve ir para a caixa.

3 · Escolher o tipo adequado

index.html: depois do solicitante, antes do comentário CAMPOS.

```
<label for="equipamento">Equipamento</label>
<input id="equipamento" name="equipamento"
  type="text" required>

<label for="contato">E-mail fictício</label>
<input id="contato" name="contato"
  type="email" required>
```



Trecho para o quadro · bloco completo no portal

Digite PC-07 e um e-mail fictício. O teste de formato será feito com o botão na etapa 8.

4 · Oferecer opções com select

index.html: após contato, antes do comentário CAMPOS.

```
<label for="categoria">Categoria
(obrigatório)</label>
<select id="categoria" name="categoria" required>
  <option value="">Escolha uma opção</option>
  <option value="hardware">Hardware</option>
  <option value="software">Software</option>
  <option value="rede">Rede</option>
  <option value="outro">Outro</option>
</select>
```



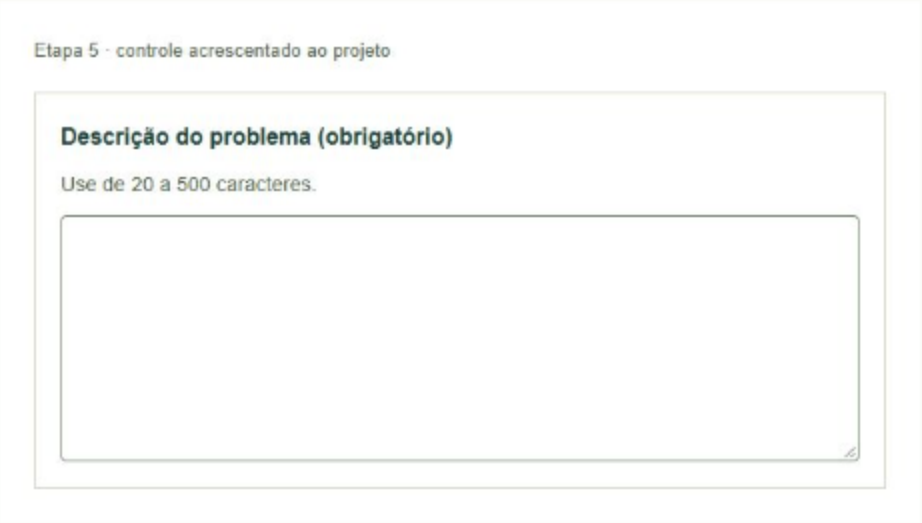
Trecho para o quadro · bloco completo no portal

Abra a lista e escolha Rede. Em 23/10, Outro é apenas uma opção; não abre campo extra.

5 · Dar espaço para a descrição

index.html: depois da categoria, antes do comentário CAMPOS.

```
<label for="descricao">Descrição do problema</label>
<p id="ajuda-descricao">20 a 500 caracteres.</p>
<textarea id="descricao" name="descricao"
  rows="5" required
  minlength="20" maxlength="500"
  aria-describedby="ajuda-descricao"></textarea>
```



Trecho para o quadro · bloco completo no portal

Escreva uma frase que descreva um defeito fictício. O limite será testado após a etapa 8.

6 · Marcar uma única prioridade

index.html: depois do fechamento de div.campos e antes do comentário GRUPOS.

```
<fieldset>
  <legend>Prioridade (obrigatória)</legend>
  <label><input type="radio" name="prioridade"
    value="normal" required> Normal</label>
  <label><input type="radio" name="prioridade"
    value="alta"> Alta</label>
</fieldset>
```



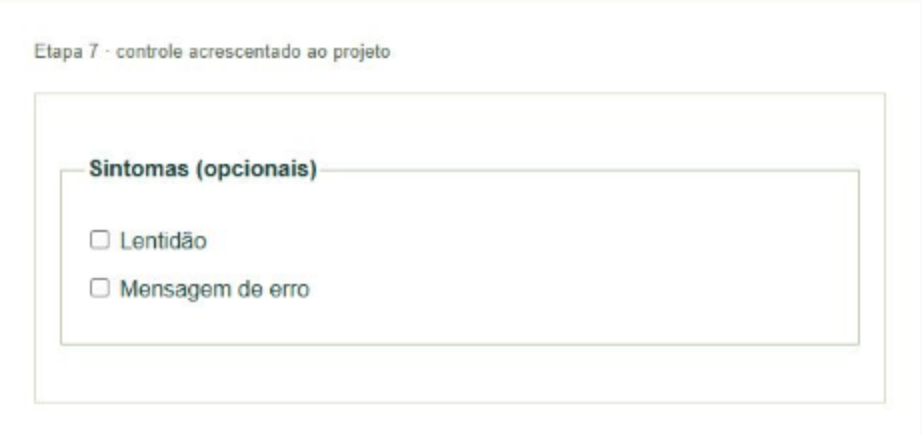
Trecho para o quadro · bloco completo no portal

Marque Normal e depois Alta: a primeira deve ser desmarcada.

7 · Aceitar mais de um sintoma

index.html: após o fieldset de prioridade e antes do comentário GRUPOS.

```
<fieldset>
  <legend>Sintomas (opcionais)</legend>
  <label><input type="checkbox" name="sintomas"
    value="Lentidão"> Lentidão</label>
  <label><input type="checkbox" name="sintomas"
    value="Mensagem de erro">
    Mensagem de erro</label>
</fieldset>
```



Trecho para o quadro · bloco completo no portal

Marque as duas; depois desmarque uma. Nenhum sintoma é obrigatório.

8 · Testar o formulário

index.html: substitua o comentário BOTÃO, ainda dentro de form, depois dos grupos.

```
<div class="acoes">
  <button id="simular" type="submit" disabled>
    Testar campos
  </button>
</div>
```



Trecho para o quadro · bloco completo no portal

Teste vazio, e-mail inválido e dados válidos. O resultado válido é uma mensagem local, sem resumo ainda.

Conferir a estrutura construída

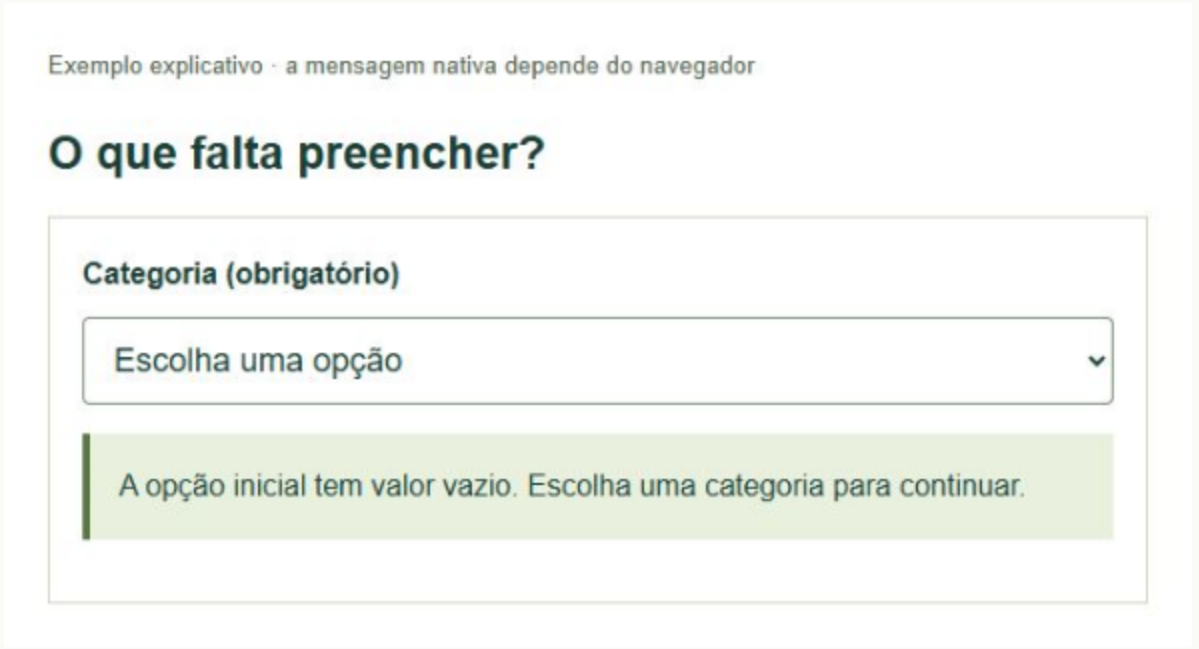
```
<main>
  <form id="formulario">
    <div class="campos">
      <!-- campos de texto e lista -->
    </div>
    <!-- prioridade e sintomas -->
    <!-- botão -->
  </form>
  <p id="status" role="status"></p>
</main>
```

Os campos ficam dentro de div.campos; os dois fieldsets vêm depois dessa div.
Tudo, inclusive o botão, fica dentro de um único form.

Por que o navegador bloqueou?

required exige preenchimento;
type="email" confere o formato.

A descrição exige 20 a 500 caracteres.
Essas regras não provam que os dados são verdadeiros.



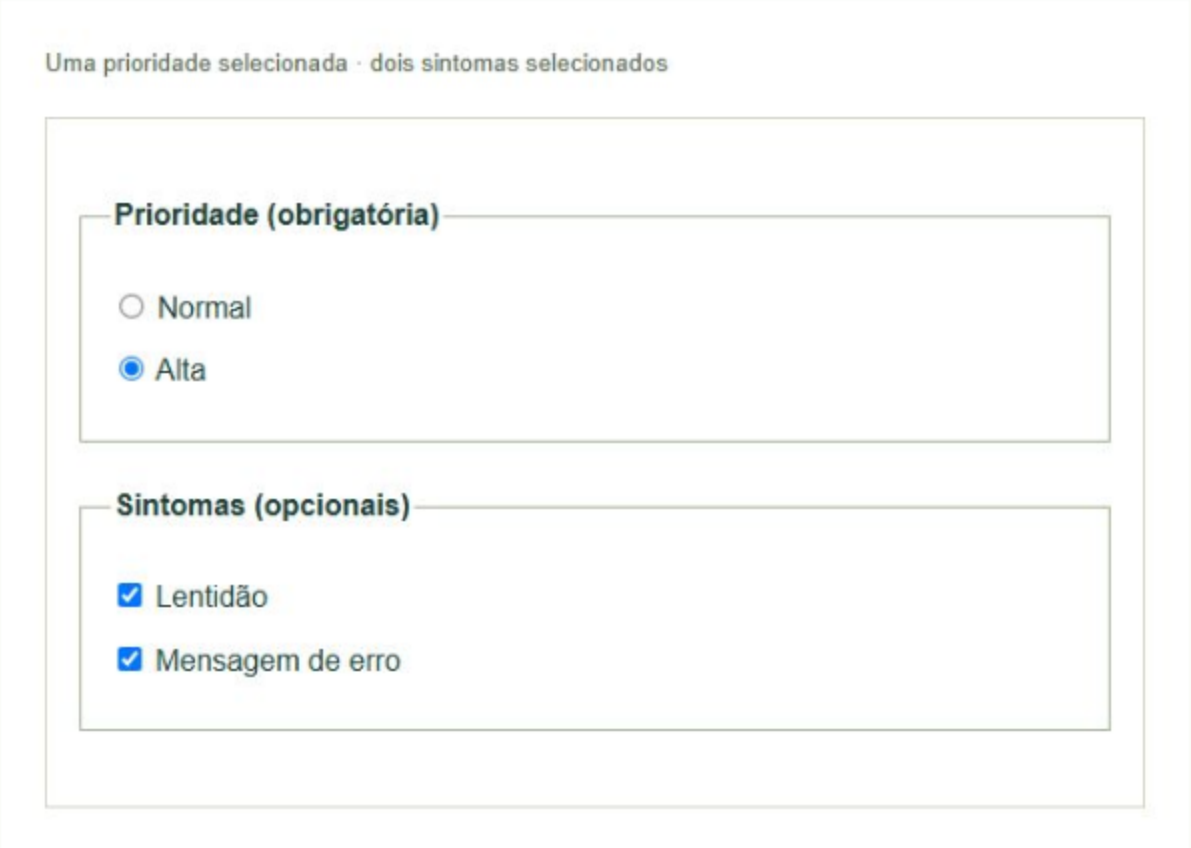
Conferência dos testes

Caso	Resultado esperado
Campos obrigatórios vazios	O navegador impede a simulação.
Contato sem formato de e-mail	O campo de contato fica inválido.
Descrição digitada com menos de 20 caracteres	O navegador pede completar o texto.

Radio ou checkbox?

Prioridade: só uma pode ficar marcada.

Sintomas: duas, uma ou nenhuma podem ficar marcadas.



O nome do dado

for="contato"

id="contato"

name="contato"

chave do dado

id liga o rótulo ao campo. name identifica o dado que será coletado.
Nas próximas aulas, FormData vai ler esses nomes. Hoje confira o HTML.

Agora: conferir sem copiar o modelo

Caso	Resultado esperado
Clique no rótulo do equipamento	O foco vai para o campo correspondente.
Marcar duas prioridades em sequência	Só a última permanece marcada.
Marcar os dois sintomas	As duas opções permanecem marcadas.
Preenchimento válido	Mensagem local; nenhuma navegação de envio.

Registro e entrega

No portal, abra Registrar testes na aula 02 e baixe o TXT identificado.

Entregue o projeto em ZIP com os três arquivos e o TXT durante a aula. Não há envio automático.

Uma explicação para fechar

> Qual é o papel de name? Qual teste mostrou que seu formulário estava funcionando?

> Mostre no seu HTML uma tag, um atributo e um teste que você realizou.

O mesmo projeto continua

23/10 · HTML

Campos e regras

→

30/10 · CSS

Layout e foco

→

06/11 · JavaScript

Coleta e resumo

Guarde sua pasta: em 30/10 vamos melhorar o layout com CSS.
Não exigimos confirmação de e-mail, resumo ou campo condicional hoje.

Um formulário legível em qualquer tela

CSS, responsividade, instruções e navegação pelo teclado
Continue o chamado de suporte.

Observe → construa → salve → teste → explique

O que vamos construir

CSS, responsividade, instruções e navegação pelo teclado

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Organizar em uma coluna

Na janela estreita, os campos cabem sem rolagem horizontal.

→

2 · Acrescentar duas colunas

Na janela larga, duas colunas; a descrição ocupa a linha inteira.

→

3 · Destacar o foco

O campo em foco tem contorno visível e segue a ordem do HTML.

Siga esta ordem; reteste a cada alteração.

Exemplo: Layout começa pela tela estreita

style.css: confira as regras de .campos e .campo.

```
* { box-sizing: border-box; }
.campos {
  display: grid;
  gap: 1.2rem;
}
.campo { min-width: 0; }
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Duas colunas quando há espaço

style.css: no final, acrescente a media query de 48rem.

```
@media (min-width: 48rem) {
  .campos { grid-template-columns: 1fr 1fr; }
  .largo { grid-column: 1 / -1; }
}
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Foco visível e ordem de leitura

style.css: acrescente :focus-visible antes da media query.

```
:focus-visible {
  outline: 3px solid #6a8f32;
  outline-offset: 4px;
}
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Teste da experiência

> Redimensione para 360, 768 e 1280 pixels. Há corte ou rolagem horizontal?

> Use Tab, Shift+Tab e 200% de zoom. Você enxerga e alcança cada controle?

Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> A base de apoio deve ser usada apenas se precisar recuperar o ponto de partida.

Conferência dos testes

Caso	Resultado esperado
Tela de aproximadamente 360px	Uma coluna; controles sem corte lateral.
Tela larga a partir de 48rem	Duas colunas; descrição ocupa a linha.
Tab e Shift+Tab	Foco visível, sequência compreensível.
Ampliação de 200%	Conteúdo legível e controles alcançáveis.
Texto longo na instrução	Quebra de linha sem sobrepor outro campo.

Conferência dos testes (continuação)

Caso	Resultado esperado
Preenchimento inválido	Regras HTML da aula anterior continuam ativas.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Qual decisão ajudou a tela estreita? Como você conferiu o foco?

> Guarde a versão desta aula: ela será o ponto de partida para a continuidade.



Oficina de formulários

Digite no HTML. Teste cada controle.
Envie e veja os dados na própria página.

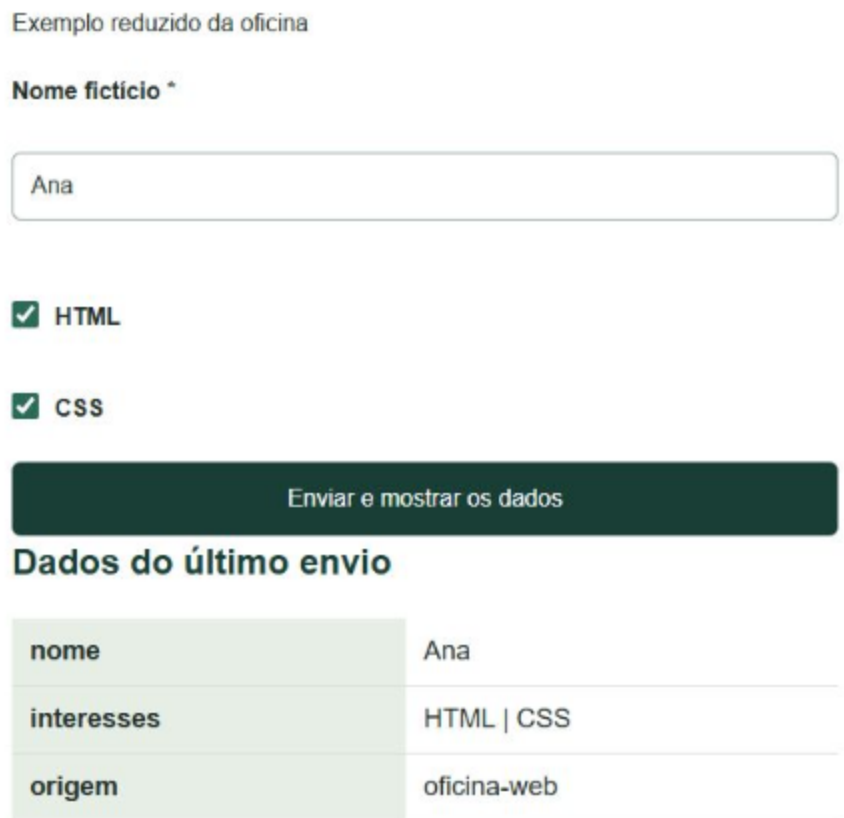
HTML para digitar · CSS e JavaScript prontos

O resultado da oficina

Um formulário com diferentes controles.

Um resumo abaixo, com as chaves e os valores.

A página continua aberta para novos testes.



INSTITUTO FEDERAL
Pará
Campus Breves

Formulário, tag e tipo

form agrupa os controles e define o envio.

input é uma tag; type escolhe seu comportamento.

select e textarea são outras tags de formulário.

```
<form id="formulario">
  <input type="text" name="nome">
</form>
```

INSTITUTO FEDERAL
Pará
Campus Breves

Abrir o projeto no VS Code

Baixe aula-04-base.zip no portal e extraia.

Arquivo → Abrir Pasta → escolha a pasta extraída.

Abra index.html com Open with Live Server.

Abra o HTML no editor e mantenha o navegador ao lado.

INSTITUTO FEDERAL
Pará
Campus Breves

Onde digitar

A base começa sem campos nem botões.

Substitua somente cada comentário ETAPA pelo código.

Etapas 01–17: #campos. Etapas 18–20: #acoes.

Preserve os ids formulario, resultado, linhas e status.

```
<!-- ETAPA 01 · Texto e identificação do campo -->
<!-- ETAPA 02 · Busca -->
<!-- Continue nos comentários seguintes. -->
```

Texto e identificação do campo

index.html · substitua somente o comentário ETAPA 01, dentro de #campos.

```
<label for="nome">Nome fictício *</label>
<input type="text" id="nome" name="nome"
  required minlength="3" autocomplete="off">
```



label identifica o campo; for aponta para o id único.

name é a chave do dado. type="text" aceita uma linha.

required e minlength="3" pedem pelo menos 3 caracteres.

SALVE E TESTE · Clique no rótulo: o cursor deve ir para o campo. Digite Ana.

Busca

index.html · substitua somente o comentário ETAPA 02, dentro de #campos.

```
<label for="busca">O que você quer pesquisar?</label>
<input type="search" id="busca" name="busca">
```



search representa uma busca. O visual pode parecer com text.

O campo sozinho não pesquisa: isso depende da aplicação.

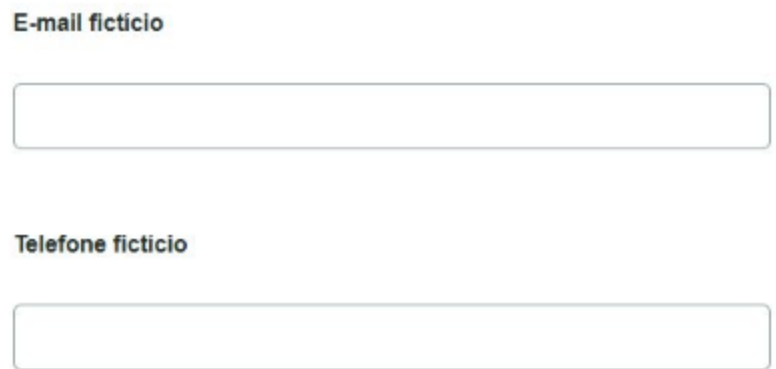
SALVE E TESTE · Digite teclado. Observe se o navegador oferece um ícone para limpar.

Contato: e-mail e telefone

index.html · substitua somente o comentário ETAPA 03, dentro de #campos.

```
<label for="email">E-mail fictício</label>
<input type="email" id="email" name="email">

<label for="telefone">Telefone fictício</label>
<input type="tel" id="telefone" name="telefone">
```



email verifica o formato quando preenchido.

tel pode sugerir teclado telefônico no celular.

tel sozinho não exige um formato de telefone.

SALVE E TESTE · Compare ana com ana@example.com no e-mail. Em tel, experimente letras.

Endereço web e senha

index.html · substitua somente o comentário ETAPA 04, dentro de #campos.

```
<label for="site">Site de exemplo</label>
<input type="url" id="site" name="site">

<label for="senha">Senha inventada</label>
<input type="password" id="senha" name="senha">
```



url espera um endereço absoluto, como https://example.com.

password esconde os caracteres na tela.

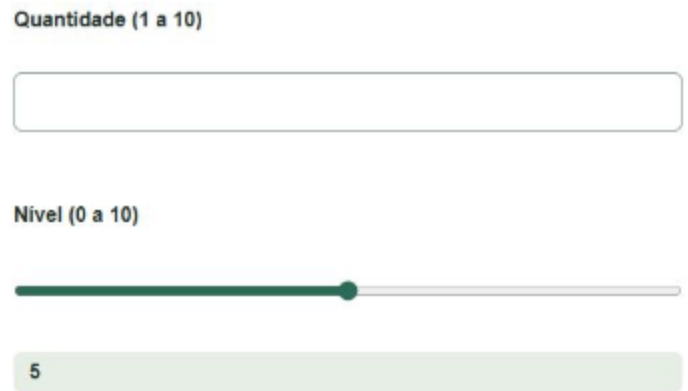
Use uma senha inventada. O resumo deste projeto a mascara.

SALVE E TESTE · Digite https://example.com. Na senha, digite abc123 e observe a máscara.

Número e intervalo

index.html · substitua somente o comentário ETAPA 05, dentro de #campos.

```
<label for="quantidade">Quantidade (1 a 10)</label>
<input type="number" id="quantidade"
  name="quantidade" min="1" max="10" step="1">
<label for="nivel">Nível (0 a 10)</label>
<input type="range" id="nivel" name="nivel"
  min="0" max="10" value="5">
<output for="nivel" id="valor-nivel">5</output>
```



min e max limitam os valores; step define o incremento.

number serve para quantidades, não para telefone ou código.

O JS pronto atualiza o output ao mover o range.

SALVE E TESTE · Digite 11 na quantidade. Mova o controle e confira o número ao lado.

Data e horário

index.html · substitua somente o comentário ETAPA 06, dentro de #campos.

```
<label for="data">Data do atendimento</label>
<input type="date" id="data" name="data">

<label for="hora">Horário preferido</label>
<input type="time" id="hora" name="hora">
```



date recebe uma data; time recebe um horário.

A aparência depende do navegador e do sistema.

No resumo, a data usa AAAA-MM-DD e a hora usa HH:MM.

SALVE E TESTE · Escolha 06/11/2026 e 09:30. Depois do envio, compare os formatos.

Data e hora no mesmo controle

index.html · substitua somente o comentário ETAPA 07, dentro de #campos.

```
<label for="agendamento">Agendamento</label>
<input type="datetime-local" id="agendamento"
  name="agendamento">
```



datetime-local reúne data e horário.

O valor não inclui fuso horário.

Exemplo de valor: 2026-11-06T09:30.

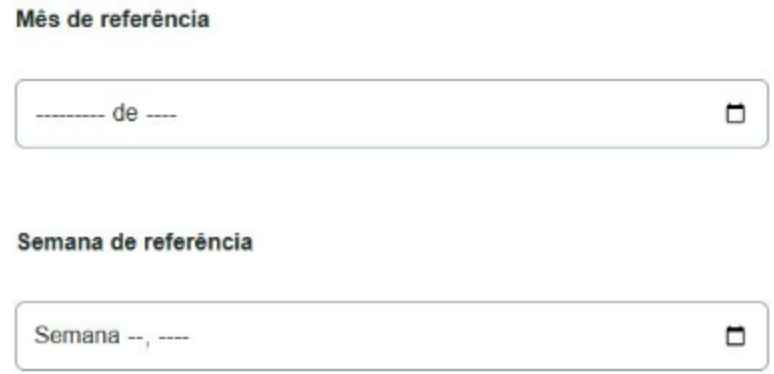
SALVE E TESTE · Escolha uma data e uma hora; procure o T no resumo após enviar.

Mês e semana

index.html · substitua somente o comentário ETAPA 08, dentro de #campos.

```
<label for="mes">Mês de referência</label>
<input type="month" id="mes" name="mes">

<label for="semana">Semana de referência</label>
<input type="week" id="semana" name="semana">
```



month usa ano e mês, como 2026-11.

week usa ano e semana ISO, como 2026-W45.

O seletor visual pode variar ou virar um campo de texto.

SALVE E TESTE · Escolha um mês e uma semana e compare seus valores no resumo.

Cor

index.html · substitua somente o comentário ETAPA 09, dentro de #campos.

```
<label for="cor">Cor do tema</label>
<input type="color" id="cor" name="cor"
  value="#183e35">
```



color abre um seletor de cores.

Neste exemplo, o valor é um código hexadecimal.

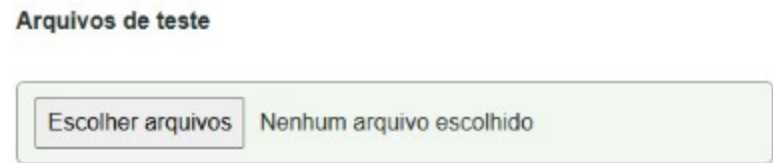
value define a cor inicial, retomada pelo botão Limpar.

SALVE E TESTE · Escolha outra cor e confira seu código no resumo.

Seleção de arquivos

index.html · substitua somente o comentário ETAPA 10, dentro de #campos.

```
<label for="anexos">Arquivos de teste</label>
<input type="file" id="anexos" name="anexos"
  multiple accept=".txt,.png,.jpg,.pdf">
```



multiple permite escolher mais de um arquivo.

accept sugere formatos no seletor; não garante o conteúdo.

O resumo mostra nome e tamanho; não faz upload.

SALVE E TESTE · Escolha dois arquivos fictícios. Confira se os dois aparecem após enviar.

Uma escolha: radio

index.html · substitua somente o comentário ETAPA 11, dentro de #campos.

```
<fieldset>
  <legend>Turno</legend>
  <label><input type="radio" name="turno"
    value="Manhã"> Manhã</label>
  <label><input type="radio" name="turno"
    value="Tarde"> Tarde</label>
</fieldset>
```



O mesmo name forma um grupo de radios.

value informa qual opção foi escolhida.

O label envolve o input: a associação é implícita.

SALVE E TESTE · Marque Manhã e depois Tarde. Apenas uma deve permanecer marcada.

Várias escolhas: checkbox

index.html · substitua somente o comentário ETAPA 12, dentro de #campos.

```
<fieldset>
  <legend>Interesses</legend>
  <label><input type="checkbox" name="interesses"
    value="HTML"> HTML</label>
  <label><input type="checkbox" name="interesses"
    value="CSS"> CSS</label>
</fieldset>
```



Checkboxes podem ser marcados independentemente.

O mesmo name pode produzir vários valores.

Opções desmarcadas não entram no FormData.

SALVE E TESTE · Marque HTML e CSS. As duas escolhas devem aparecer no resumo.

Lista de opções: select

index.html · substitua somente o comentário ETAPA 13, dentro de #campos.

```
<label for="assunto">Assunto</label>
<select id="assunto" name="assunto">
  <option value="">Escolha uma opção</option>
  <option value="hardware">Hardware</option>
  <option value="software">Software</option>
</select>
```



select cria uma lista; option cria cada alternativa.

O dado enviado vem de value.

O resumo pode mostrar hardware, mesmo que o rótulo seja Hardware.

SALVE E TESTE · Escolha Hardware e compare o texto da opção com o valor mostrado.

Lista com várias seleções

index.html · substitua somente o comentário ETAPA 14, dentro de #campos.

```
<label for="oficinas">Oficinas</label>
<select id="oficinas" name="oficinas"
  multiple size="3">
  <option value="HTML">HTML</option>
  <option value="CSS">CSS</option>
  <option value="JavaScript">JavaScript</option>
</select>
```



multiple também funciona em select.

No Windows, use Ctrl + clique para escolhas separadas.

As seleções entram com o mesmo name.

SALVE E TESTE · Selecione HTML e CSS com Ctrl + clique. Confira as duas no resumo.

Texto com várias linhas: textarea

index.html · substitua somente o comentário ETAPA 15, dentro de #campos.

```
<label for="mensagem">Mensagem</label>
<textarea id="mensagem" name="mensagem"
  rows="3" maxlength="200"></textarea>
```



textarea recebe texto com quebras de linha.

Ela tem tag de fechamento; input não tem.

Deixe abertura e fechamento juntos para começar vazia.

SALVE E TESTE · Digite duas linhas. Use teste e confira se aparece como texto.

Sugestões: datalist

index.html · substitua somente o comentário ETAPA 16, dentro de #campos.

```
<label for="cidade">Cidade</label>
<input type="text" id="cidade" name="cidade"
  list="cidades">
<datalist id="cidades">
  <option value="Breves"></option>
  <option value="Belém"></option>
</datalist>
```



list aponta para o id do datalist.

A lista sugere valores, mas aceita outro texto.

O suporte e a experiência de uso variam entre navegadores.

SALVE E TESTE · Escolha Breves; depois digite outra cidade que não esteja na lista.

INSTITUTO FEDERAL
Pará
Campus Breves

Dado sem controle visível: hidden

index.html · substitua somente o comentário ETAPA 17, dentro de #campos.

```
<input type="hidden" name="origem"
  value="oficina-web">
```

Na tela: nenhum campo visível.

Após enviar

name	Valor
origem	oficina-web

hidden não aparece como campo na tela.

Seu name e seu value entram no FormData.

Ele pode ser alterado; não serve para guardar segredos.

SALVE E TESTE · Não deve surgir um campo. Após enviar, procure origem: oficina-web.

Botões: ação e restauração

index.html · substitua somente o comentário ETAPA 18, dentro de #acoes.

```
<input type="button" value="Contar campos"
  data-acao="contar">
<input type="reset" value="Limpar">
```



button não envia. data-acao liga a ação ao JS pronto.

reset restaura os valores iniciais, inclusive cor e range.

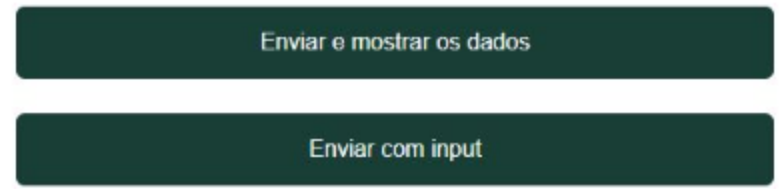
O JS também esconde o resumo e atualiza o output.

SALVE E TESTE · Conte os controles. Teste Limpar e confira range = 5 e a cor inicial.

Enviar e mostrar os dados

index.html · substitua somente o comentário ETAPA 19, dentro de #acoes.

```
<button type="submit" name="acao" value="principal">
  Enviar e mostrar os dados
</button>
<input type="submit" name="acao"
  value="Enviar com input">
```



Os dois controles disparam o envio do mesmo formulário.

O navegador verifica as regras antes do evento submit.

O resumo recebe o name/value do botão acionado.

SALVE E TESTE · Preencha Nome com Ana. Envie pelos dois botões e compare a linha acao.

Envio por imagem

index.html · substitua somente o comentário ETAPA 20, dentro de #acoes.

```
<input type="image" id="enviar-imagem"
  name="clique" alt="Enviar por imagem"
  width="140" height="42">
```



image também é um botão de envio. alt dá seu nome acessível.

O JS fornece a imagem embutida: não há um quarto arquivo.

O envio inclui as coordenadas clique.x e clique.y.

SALVE E TESTE · Clique na imagem. O resumo deve aparecer com clique.x e clique.y.

INSTITUTO FEDERAL
Pará
Campus Breves

O JavaScript que já veio pronto

submit: reage ao envio após as regras do HTML.

preventDefault: evita a navegação do envio padrão.

FormData: lê name e value, incluindo opções repetidas.

textContent: mostra o dado como texto.

```
formulario.addEventListener('submit', (evento) => {
  evento.preventDefault();
  const dados = new FormData(formulario, evento.submitter);
  // O restante do apoio monta a tabela abaixo.
});
```

Explicação do apoio pronto · não digitar outro listener

Testar regras e corrigir

Nome vazio ou com 2 letras: envio bloqueado.

E-mail preenchido como ana: envio bloqueado.

URL sem protocolo válido: corrija para https://example.com.

Quantidade 11: fora do máximo 10.

Campos opcionais vazios podem continuar vazios.

Ler os dados abaixo do formulário

Marque dois interesses e duas oficinas.

Escolha arquivos fictícios e envie pelo botão principal.

Confira interesses e oficinas com todos os valores.

Compare data, cor, origem e acao com o HTML.

Trecho do resumo

nivel	8
data	2026-11-06
anexos	arquivo-teste.txt (33 bytes) | etapa-01.png (3945 bytes)
turno	Tarde
interesses	HTML | CSS
oficinas	HTML | CSS
origem	oficina-web

INSTITUTO FEDERAL
Pará
Campus Breves

Experimentos com name e disabled

Remova temporariamente name="busca", salve e envie.

O campo continua visível, mas sua chave some do resumo.

Restaure name. Adicione disabled ao campo e repita.

Desmarque todos os interesses: a chave também não entra.

Desfaça as alterações depois de comparar.

INSTITUTO FEDERAL
Pará
Campus Breves

Conferência dos 22 tipos de input

Texto: text · search · email · tel · url · password

Valores: number · range · color

Tempo: date · time · datetime-local · month · week

Escolhas e arquivos: radio · checkbox · file

Apoio e ações: hidden · button · reset · submit · image

INSTITUTO FEDERAL
Pará
Campus Breves

Entregar a oficina

Teste reenvio: o resumo substitui o anterior.

Teste Limpar: restaura valores iniciais e esconde o resumo.

Compacte os três arquivos: aula04-seu-nome.zip.

No portal, registre os testes e baixe o TXT.

Entregue o ZIP e o TXT ao professor durante a aula.

INSTITUTO FEDERAL
Pará
Campus Breves

Explicar e continuar

Qual é a diferença entre id, name e type?

Quando você escolheria radio, checkbox ou select?

Por que o resumo mostra dados sem enviar a um servidor?

13/11: retomaremos validação no chamado de suporte.

As referências clicáveis estão na aula do portal.

Mensagens que ajudam a corrigir

Regras complementares, validade personalizada e casos de teste
Continue o chamado de suporte.

Observe → construa → salve → teste → explique

O que vamos construir

Regras complementares, validade personalizada e casos de teste

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Adicionar confirmação

O novo campo pede repetição do contato.

→

2 · Comparar e limpar mensagens

Contatos diferentes pedem correção. Ao igualar, o erro desaparece.

→

3 · Conferir antes do resumo

Espaços não completam a descrição. Uma tentativa inválida mantém os valores.

Siga esta ordem; reteste a cada alteração.

Exemplo: Comparação de dois campos

script.js: após selecionar #contato e #confirmar, antes do listener de submit.

```
function conferirContato() {
  const diferente = confirmar.value !== contato.value;
  confirmar.setCustomValidity(
    confirmar.value && diferente ? 'Repita o mesmo e-mail.' : ''
  );
}
contato.addEventListener('input', conferirContato);
confirmar.addEventListener('input', conferirContato);
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Descrição e limites do teste

script.js: após selecionar #descricao, antes do listener de submit.

```
function conferirDescricao() {
  const curta = descricao.value.length > 0
    && descricao.value.trim().length < 20;
  descricao.setCustomValidity(
    curta ? 'Descreva o problema com pelo menos 20 caracteres.' : ''
  );
}
descricao.addEventListener('input', conferirDescricao);
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Nova tentativa sem mensagens antigas

script.js: substitua o listener de submit; mantenha apenas um.

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  conferirContato();
  conferirDescricao();
  if (!form.reportValidity()) return;
  mostrarResumo(new FormData(form));
});
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> A base de apoio deve ser usada apenas se precisar recuperar o ponto de partida.

Conferência dos testes

Caso	Resultado esperado
E-mails diferentes	Mensagem de repetir o mesmo contato.
Corrigir o segundo e-mail	A mensagem antiga é limpa.
Mudar o primeiro depois da confirmação	A divergência volta a ser identificada.
Descrição formada por espaços	A regra complementar bloqueia a simulação.
Descrição digitada com 19 caracteres úteis	A simulação fica bloqueada.

Conferência dos testes (continuação)

Caso	Resultado esperado
Descrição digitada com 20 caracteres úteis	A regra de tamanho permite continuar.
Novo preenchimento após erro	Não permanece mensagem personalizada antiga.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Quando sua mensagem é limpa? Que teste demonstrou a correção do erro anterior?

> Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

O caminho dos dados

GET, POST, HTTPS e contrato do projeto integrador
Continue o chamado de suporte.

Observe → construa → salve → teste → explique

O que vamos construir

GET, POST, HTTPS e contrato do projeto integrador

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Separar simulação e servidor

O projeto continua local, sem abrir uma solicitação real.

→

2 · Representar GET e POST

O navegador mostra dois textos para comparação; não executa uma requisição.

→

3 · Conferir o contrato

A representação de POST não duplica confirmar; HTTPS protege o tráfego, não valida os dados.

Siga esta ordem; reteste a cada alteração.

Os limites do protótipo

> O resumo é gerado no navegador. Os dados não chegam a um servidor de chamados.

> Hoje vamos comparar representações e documentar uma rota futura, sem fazer o envio.

Exemplo: GET e os parâmetros da URL

script.js: trecho dentro de representarEnvio(dados). dados já é o parâmetro da função.

```
const parametros = new URLSearchParams(dados);
const exemplo = 'https://exemplo.test/chamados?'
  + parametros.toString();
// Apenas exibir o texto. Não navegar para esse endereço.
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

POST, HTTPS e conferência no servidor

> POST coloca os dados no corpo da requisição. O método sozinho não criptografa os dados.

> HTTPS protege o tráfego. O servidor ainda precisa conferir todas as regras.

Exemplo: Contrato do integrador

index.html: antes de form, registre este comentário sobre a aplicação futura.

```
<!-- Exemplo de contrato para uma aplicação futura.
Método: POST. Rota proposta: /chamados.
O servidor deve validar os campos novamente.
Este projeto estático não implementa essa rota.
-->
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> A base de apoio deve ser usada apenas se precisar recuperar o ponto de partida.

Conferência dos testes

Caso	Resultado esperado
Simulação com dados fictícios	Resumo e representações locais, sem envio.
Espaços no nome	URL de exemplo com parâmetros codificados.
Dois sintomas selecionados	Representação de POST preserva a lista.
Contrato sem indicar implementação futura	Revisar para não prometer serviço existente.
POST sem HTTPS no argumento	Explicar que POST não oferece criptografia.

Conferência dos testes (continuação)

Caso	Resultado esperado
Campo sem name	Identificar a ausência na coleta e corrigir o contrato.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Qual parte do seu projeto roda no navegador? O que ainda faltaria num serviço real?

> Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Um formulário que reage ao contexto

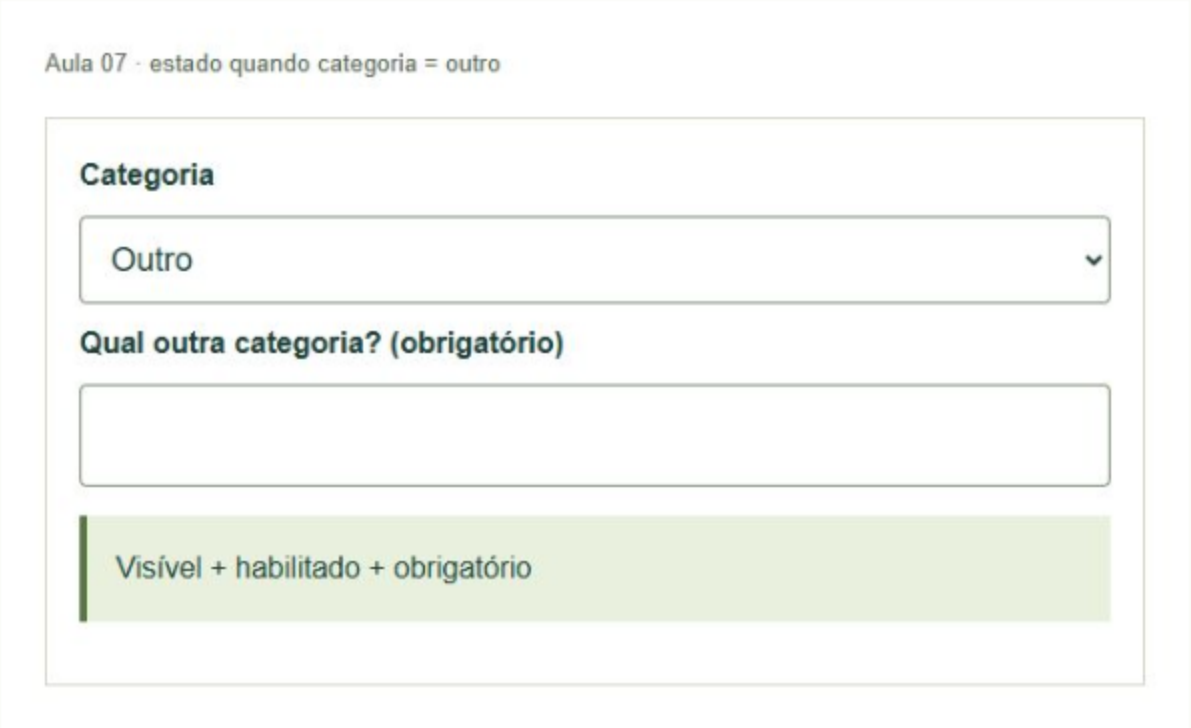
Campos condicionais, estado e organização das funções
Continue o chamado de suporte.

Observe → construa → salve → teste → explique

O que vamos construir

Campos condicionais, estado e organização das funções

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Adicionar o campo complementar

O campo começa oculto e desativado.

→

2 · Sincronizar o estado

Outro mostra e exige o campo. Hardware esconde e retira o controle da coleta.

→

3 · Repetir o ciclo completo

Nenhum valor antigo reaparece no resumo; reiniciar restaura o estado inicial.

Siga esta ordem; reteste a cada alteração.

Exemplo: A interface responde a uma escolha

script.js: após selecionar categoria, blocoOutro e outro; antes do listener de submit.

```
function atualizarOutro() {
  const ativo = categoria.value === 'outro';
  blocoOutro.hidden = !ativo;
  outro.disabled = !ativo;
  outro.required = ativo;
  if (!ativo) outro.value = '';
}
categoria.addEventListener('change', atualizarOutro);
atualizarOutro();
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Esconder não desativa o controle

script.js: após selecionar categoria, blocoOutro e outro; antes do listener de submit.

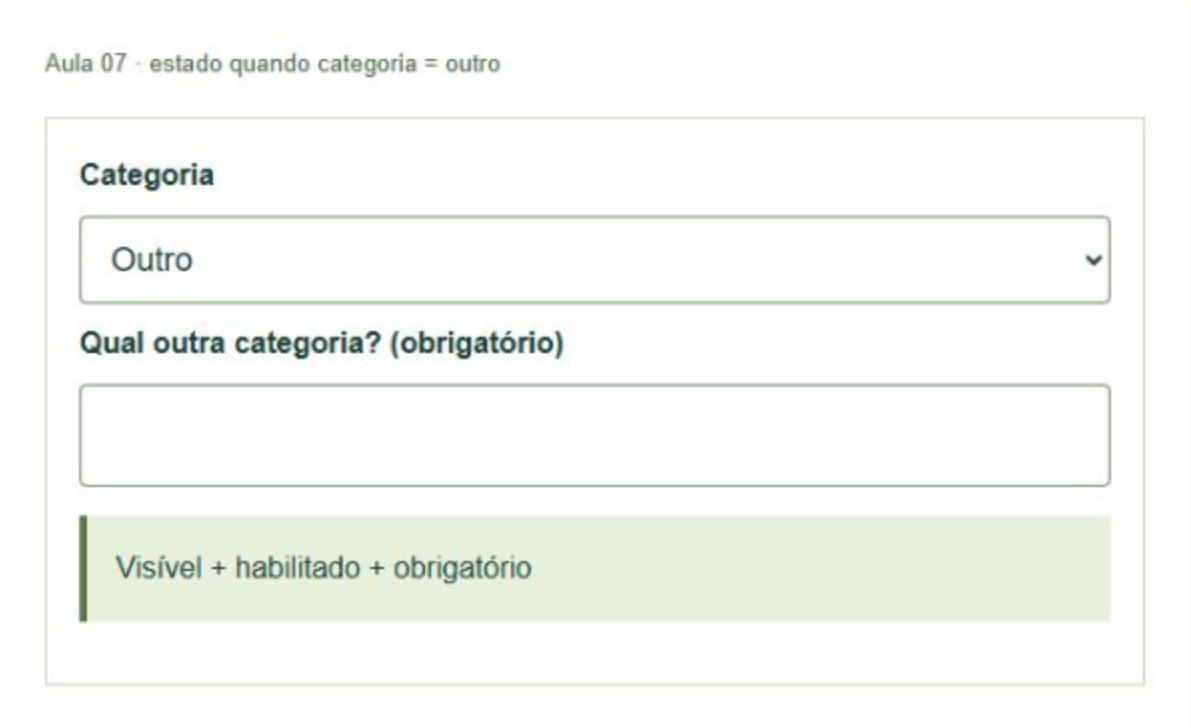
```
// hidden controla a apresentação do bloco.
// disabled retira o campo da validação e do FormData.
// required pede um valor quando o campo está ativo.
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Conferência do fluxo inteiro

Teste entrar em Outro, tentar sem preencher, informar um detalhe e gerar o resumo. Depois retorne a Hardware e confira que o detalhe não aparece na coleta.

Use Novo preenchimento para conferir reset e reaplicação do estado. Revise a comparação dos e-mails e as mensagens: acrescentar uma condição não deve quebrar as regras anteriores.



Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> A base de apoio deve ser usada apenas se precisar recuperar o ponto de partida.

Conferência dos testes

Caso	Resultado esperado
Abertura com categoria vazia	Campo adicional oculto e desabilitado.
Escolher Outro sem detalhar	Campo visível e obrigatório.
Outro com detalhe e dados válidos	Resumo inclui a outra categoria.
Trocar de Outro para Hardware	Bloco oculto e valor antigo limpo.
Gerar resumo após sair de Outro	Não inclui outra_categoria.

Conferência dos testes (continuação)

Caso	Resultado esperado
Novo preenchimento	Estado inicial reaplicado.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Que diferença existe entre esconder um campo e desabilitá-lo?

> Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Avaliação prática: inscrição em oficina

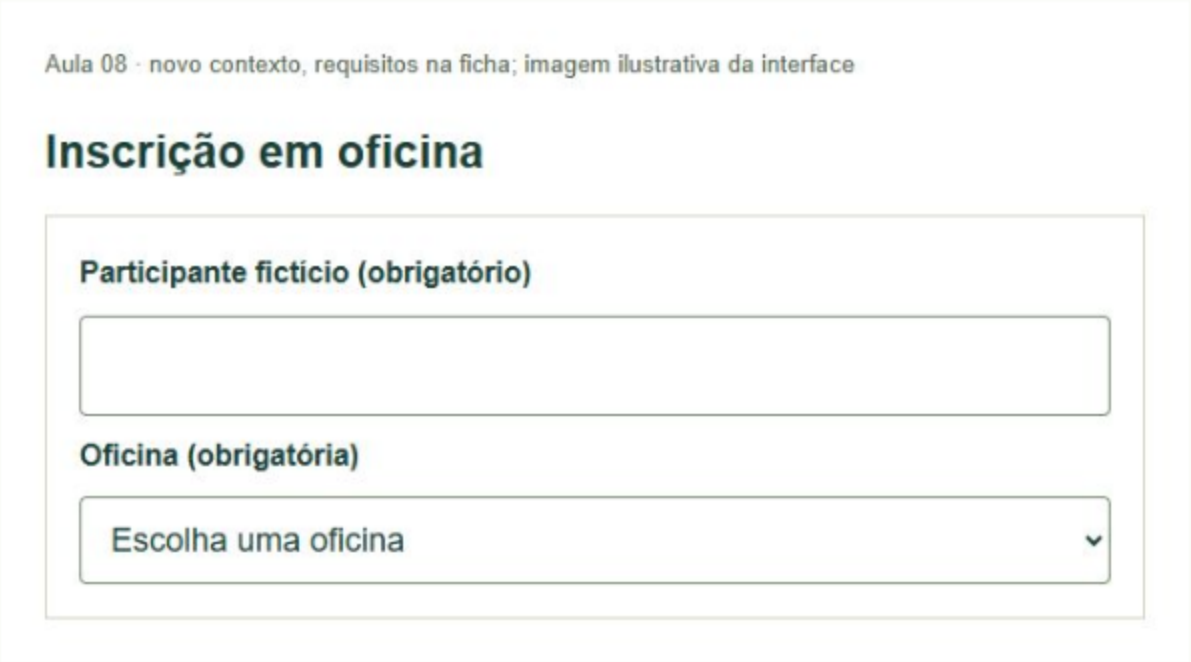
Aplicação individual dos conhecimentos de formulários
Use a nova base da atividade e demonstre o que aprendeu.

Observe → construa → salve → teste → explique

O que vamos construir

Aplicação individual dos conhecimentos de formulários

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Ler antes de implementar

Uma base nova permite demonstrar o que você aprendeu no projeto contínuo.

→

2 · Construir e testar individualmente

Entregue evidências reais; o professor considera código, comportamento e explicação.

Siga esta ordem; reteste a cada alteração.

Critérios propostos para a avaliação

Dimensão	Pontos
HTML e controles	2
CSS e acessibilidade de uso	2
Coleta e resumo local	2
Validação e estado condicional	2
Testes e explicação individual	2

Especificação dos campos

Participante: 2 a 60 caracteres. Contato e confirmação: e-mails obrigatórios e iguais.

Oficina: select obrigatório. Turno: uma escolha em radio. Recursos: escolhas opcionais.

Objetivo: 20 a 500 caracteres úteis. Outra oficina ativa e exige Tema sugerido.

Ao sair de Outra oficina, limpe e desabilite o campo complementar. Confira os nomes na ficha.

Comportamento e entrega

> Mostrar um resumo local com os dados da inscrição, preservando os dois materiais opcionais se marcados. Tratar valores como texto. Não enviar nem gravar em servidor.

> O formulário deve funcionar em tela estreita e pelo teclado. Manter valores após uma tentativa inválida. O reinício precisa limpar as mensagens e o estado condicional. Entregar os três arquivos em ZIP e o registro TXT.

Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> Use a base específica e cumpra os requisitos da ficha.

Conferência dos testes

Caso	Resultado esperado
Obrigatórios vazios	Bloqueio da tentativa.
E-mails diferentes e depois corrigidos	Erro comunicado e mensagem limpa ao corrigir.
Outra oficina sem tema	Tema obrigatório.
Trocar de Outra oficina para Web	Tema deixa a validação e a coleta.
Caderno e Notebook marcados	Resumo preserva os dois materiais.

Conferência dos testes (continuação)

Caso	Resultado esperado
Texto com sinais de tag no objetivo	Aparece como texto literal.
Tela estreita e teclado	Campos alcançáveis e foco visível.
Novo preenchimento	Regras e estado voltam à condição inicial.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Qual requisito você demonstrou com maior autonomia? Qual precisa corrigir?

> Guarde o projeto e o registro para a devolutiva e as correções de 18/12.

Corrigir, recuperar e concluir

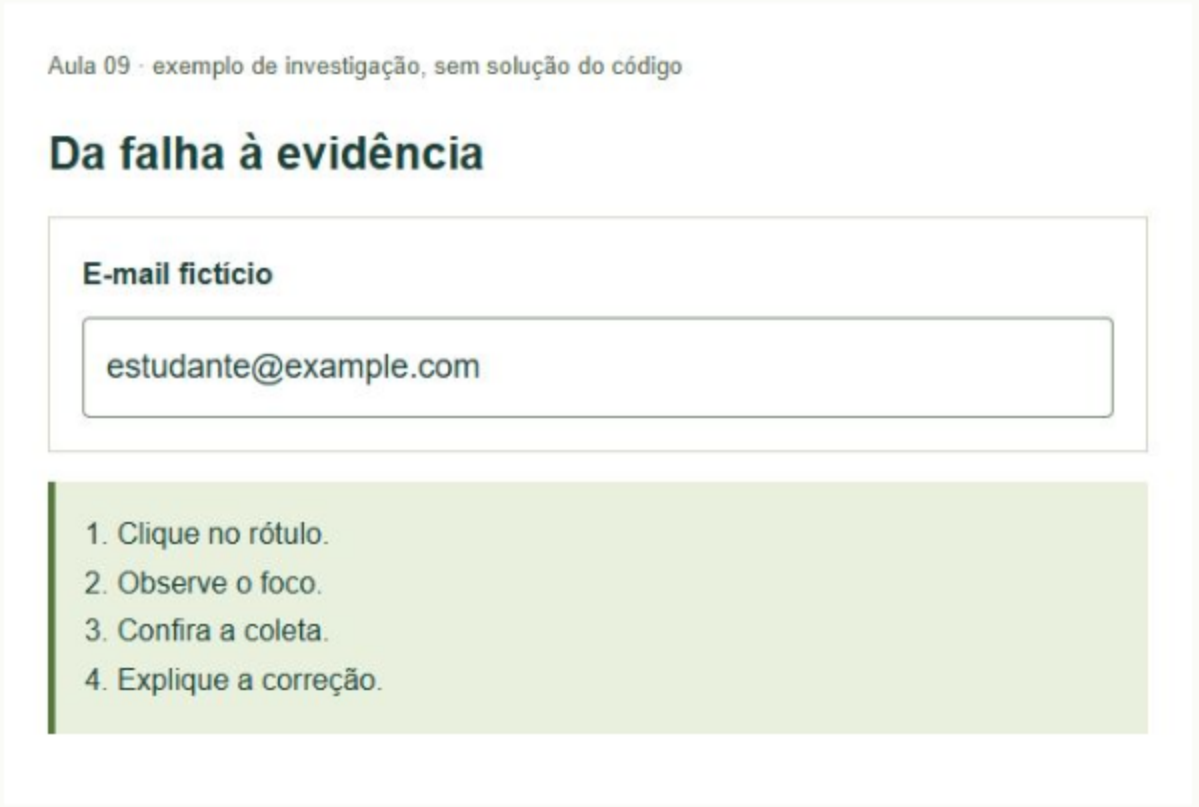
Devolutiva, depuração e nova evidência de aprendizagem
Use a nova base da atividade e demonstre o que aprendeu.

Observe → construa → salve → teste → explique

O que vamos construir

Devolutiva, depuração e nova evidência de aprendizagem

Relacione o resultado visível às alterações desta aula.



Mapa da aula

1 · Investigar antes de alterar

Cada correção tem um teste de regressão; aparência sozinha não comprova funcionamento.

→

2 · Corrigir e demonstrar

Entregue ZIP e TXT com a correção explicada; combine a evidência com a devolutiva individual.

Siga esta ordem; reteste a cada alteração.

Da devolutiva ao plano de correção

> Leia a devolutiva e identifique os requisitos que ainda precisam de evidência.

> Na base de recuperação há defeitos intencionais. Reproduza, corrija e repita o teste.

Exemplo: Investigação de rótulos e coleta

Abra a base de recuperação de 18/12 em uma pasta separada.

```
const dados = new FormData(form);
const contato = dados.get('contato');
const sintomas = dados.getAll('sintomas');
// getAll preserva várias opções com o mesmo name.
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Exemplo: Estado e nova tentativa

Ajuste os arquivos da recuperação e reteste os requisitos pendentes da avaliação.

```
// hidden controla a apresentação do bloco.
// disabled retira o campo da validação e do FormData.
// required pede um valor quando o campo está ativo.
```

Ajuste o trecho no local indicado. Salve e confira o comportamento antes de avançar.

Atividade no VS Code

> Agora altere sua versão seguindo as etapas da aula.

> Use a base específica e cumpra os requisitos da ficha.

Conferência dos testes

Caso	Resultado esperado após a correção
Clicar no rótulo de contato	O foco chega ao contato.
Preencher contato e gerar resumo	O valor informado aparece no resumo.
Escolher Outra oficina	Tema fica habilitado e obrigatório.
Sair de Outra oficina	Tema fica oculto e fora da coleta.
Corrigir um valor inválido	A próxima tentativa pode ocorrer.

Conferência dos testes (continuação)

Caso	Resultado esperado após a correção
Repetir o caso da avaliação	A evidência do critério pendente fica registrada.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

> Qual era a causa do erro, como você a corrigiu e qual teste confirmou a mudança?

> Guarde o código e o registro como evidências do que você aprendeu neste período.